Lista Teórica E3

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Aula E3 — NLP + Classificação (Aplicação)

Exercício 1 — do texto à matriz. Considere três documentos:

d_1 = "o gato dorme", d_2 = "o cachorro dorme", d_3 = "o gato corre e o cachorro corre".

(a) Monte o vocabulário em ordem alfabética e escreva a matriz documento–termo de **contagens** ($3 \times V$).

(b) O `scikit-learn` calcula $\text{idf}(t) = \ln(\frac{1+n}{1+\text{df}(t)}) + 1$, onde $\text{df}(t)$ é o número de documentos que contêm t. Calcule o idf de *gato* e de *o*.

(c) Calcule o vetor TF-IDF de d_1, normalizado em norma ℓ_2. *Dados:* $\ln(4/3) \approx 0{,}2877$.

(d) Por que o termo *o* recebe o menor peso possível? E por que ainda assim ele não é zerado por essa fórmula?

Exercício 2 — naive Bayes multinomial e a suavização de Laplace. Um filtro de spam foi treinado sobre um vocabulário de $V = 5$ palavras, com as contagens totais abaixo (soma sobre todas as mensagens de cada classe):

	grátis	clique	oferta	reunião	amanhã	total
spam	8	5	4	0	1	18
não-spam	1	0	1	6	7	15

As prioris estimadas são $\mathbb{P}(\text{spam}) = 0{,}4$ e $\mathbb{P}(\text{não-spam}) = 0{,}6$. O modelo multinomial estima

$$\mathbb{P}(w \mid c) \;=\; \frac{N_{wc} + \alpha}{N_c + \alpha V},$$

com $\alpha = 1$ (suavização de Laplace).

(a) Classifique a mensagem "clique oferta grátis", comparando as log-posterioris das duas classes. *Dados:* $\ln 0{,}4 = -0{,}9163$, $\ln 0{,}6 = -0{,}5108$.

(b) O que aconteceria com a log-posteriori de não-spam se $\alpha = 0$ (sem suavização)? Isso é razoável?

(c) Por que se trabalha com *logaritmos* das probabilidades, e não com as probabilidades diretamente?

(d) O modelo supõe que as palavras são independentes dentro de cada classe. Cite duas maneiras óbvias pelas quais isso é falso em texto. Por que o método funciona mesmo assim?

Exercício 3 — esparsidade, e o que fazer com ela. Numa coleção de 5 572 mensagens de SMS, o `CountVectorizer` produz um vocabulário de 8 672 termos, e a matriz resultante tem 73 916 entradas não nulas.

(a) Calcule a densidade da matriz. Se ela fosse guardada densamente, em *float* de 8 bytes, quanto ocuparia? E num formato esparso que guarda, para cada não-zero, o valor (8 bytes) e o índice da coluna (4 bytes)?

(b) Por que a matriz é tão esparsa? A mediana de palavras distintas por mensagem é 11; use isso para prever a densidade e compare com o valor do item (a).

(c) O que faz `CountVectorizer(min_df=5)`? Que efeito isso tem sobre o tamanho do vocabulário, e por que costuma *ajudar* o classificador?

(d) Cite um método do curso que **não** funcionaria bem sobre uma matriz esparsa desse tipo, e explique por quê.

Exercício 4 — n-gramas e a explosão do vocabulário. O `CountVectorizer` aceita `ngram_range=(1,2)`, que acrescenta ao vocabulário todos os **pares de palavras consecutivas** além das palavras isoladas.

(a) Para os três documentos do Exercício 1, liste os bigramas de cada um. Quantos bigramas distintos existem ao todo?

(b) Que informação os bigramas recuperam, que o saco de palavras havia jogado fora? Dê um exemplo em que isso muda a classificação.

(c) Um corpus com V palavras distintas tem no máximo V^2 bigramas possíveis, mas na prática o número observado é muito menor. Por quê?

(d) Acrescentar bigramas aumenta muito o número de colunas. Isso agrava o problema do Exercício 3(d)? E o risco de superajuste?